

M. Tech Final Project Report
on
REAL-TIME SIGN LANGUAGE RECOGNITION
SYSTEM USING DEEP LEARNING AND
COMPUTER VISION

Submitted in partial fulfillment of the requirements for the award of the
degree
of

MASTER OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by
Name: Sumanpreet Kaur
(U.R.No.:2461612)

June, 2025

Prof.Manjot Kaur Gill
Assistant Professor
Department of CSE



Guru Nanak Dev Engineering College, Ludhiana-141006
IKG Punjab Technical University, Jalandhar

CERTIFICATE

This is to certify that the work presented in the Project entitled “Real-Time Sign Language Recognition using Deep Learning and Computer Vision” submitted in partial fulfillment of the degree of Master of Technology in Computer Science and Engineering at the Department of CSE, Guru Nanak Dev Engineering College, Ludhiana, to the IKG Punjab Technical University, Jalandhar, is an authentic work. The work has been undertaken under the guidance of Prof. Manjot Kaur Gill, Assistant Professor, Department of CSE.

The matter embodied in this project has not been submitted earlier for the award of any degree, to the best of my knowledge and belief.

Signature of the Student

URN: 2461612

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Signature of the Guide

Prof. Manjot Kaur Gill

Designation: Assistant Professor

Department: Computer Science and Engineering

ACKNOWLEDGEMENT

I would like to use this moment to extend my sincerest appreciation to all those individuals who have made it possible for me to complete my project “Real-Time Sign Language Recognition using Deep Learning and Computer Vision” Above all, I would also like to express my heartfelt gratitude to my project guide, Prof. Manjot Kaur Gill, for her unwavering support, motivation, and rich insights during the course of the project. Her valuable guidance, thoughtful feedback, and calm mentoring were crucial in shaping this work and assisted me in surmounting many hurdles during the research phase. I am also indebted to the Department of Computer Science and Engineering and the teachers of Guru Nanak Dev Engineering College for allowing me to pursue this project in a healthy and inspiring academic atmosphere and providing the facilities and resources essential for its conduct, which served as a strong support in making my project process efficient. My sincere appreciation is also extended to my colleagues and friends for their useful recommendations, debates, and moral encouragement, which kept me going during this process. I would also like to appreciate my family for their constant support and belief in me, which motivated me to excel. This project has been a learning experience, and I am sincerely thankful to all who helped in making it a success.

LIST OF FIGURES/TABLES

Table 1:Sign language 3 major components	1
Table 2 : Comparative Performance Analysis of MobileNetV2 and ResNet-50	32
Figure 1: Hand Gestures	2
Figure2:ResNet-50Architecture	20
Figure 3:Skip Connection	22
Figure 4: MobileNetv2	23
Figure 5:Flowchart	25
Figure 6:Model accuray comparison	29
Figure 7:Confusion matric mobileNetV2	29
Figure 8:Confusion matric ResNet50	30
Figure 9: MobileNetV2 Model accuracy	31
Figure 10:ResNet50 :Model accuracy	32

ABSTRACT

Sign language recognition plays a crucial role in bridging communication gaps between the deaf and hearing communities. The primary goal of this project is to develop a real-time system for recognizing sign language gestures using deep learning and computer vision techniques. The system aims to translate sign language into text and speech, enabling efficient communication. A Convolutional Neural Network (CNN) model, specifically ResNet50, is employed for gesture recognition, using OpenCV for real-time image processing. The system processes hand gestures and classifies them into predefined categories, including letters and numbers, using a custom dataset. Preprocessing techniques such as grayscale conversion, gamma correction, CLAHE, Gaussian blur, resizing, and normalization are applied to enhance the quality of input images and improve model accuracy. The results show that the model effectively recognizes sign language gestures with high accuracy in real-time. This work demonstrates the potential of deep learning and computer vision in developing accessible tools for the deaf community, contributing to breaking communication barriers and facilitating inclusive interactions.

Keywords: Sign Language Recognition, Deep Learning, Computer Vision, CNN, ResNet50, Real-Time Processing, Gesture Recognition, Text-to-Speech, Accessibility, OpenCV

CONTENTS

CERTIFICATE	i
ACKNOWLEDGEMENT	ii
LIST OF FIGURES/TABLES	iii
ABSTRACT	iv
CONTENTS	v
Chapter 1 Introduction	1-3
1.1 Overview	1
1.2 Background	2
1.3 Problem formulation	3
Chapter 2 Literature Review	4-12
2.1 Review of literature	4-10
2.2 Research Gap/Motivation of work	11
2.2.1 Research Gap	11
2.2 .2 Motivation of work	12
Chapter 3 Present Work	13-27
3.1 Problem Formulation	13-14
3.2 Objectives	15
3.3 Methodology	16-25
3.4 Platform Used	26-27
Chapter 4 Results and conclusions	28-35
4.1 Results	28
4.2 Conclusions	35
REFERENCES	36-38

Chapter 1

Introduction

1.1 Overview

American Sign Language (ASL) is one of the most widely used sign languages. Since the primary challenge faced by Deaf and Mute (referred to as D&M) individuals is related to communication, and they cannot rely on spoken language, their primary mode of interaction is through sign language. Communication involves the exchange of thoughts and messages through various means such as speech, signals, behavior, and visuals. D&M individuals use their hands to form gestures that convey their ideas and emotions. These gestures, which serve as non-verbal messages, are perceived visually. This form of communication is known as sign language.

Sign language is a complete language that uses visual gestures instead of spoken words. It combines elements such as hand shapes, hand orientation, movements of the hands and arms, facial expressions, and lip patterns to convey meaning. Contrary to the common misconception, sign language is not universal and varies from region to region.

Table 1: Three major components of Sign language

Fingerspelling	Word level sing vocabulary	Non-manual features
Used to spell words letter by letter	Used for the majority of communication	Facial expressions and tongue,mouth and body position

Minimizing the verbal exchange gap among D&M and non-D&M people turns into a desire to make certain effective communication among all. Sign language translation is among one of the most growing lines of research and it enables the most natural manner of communication for those with hearing impairments. A hand gesture recognition system offers an opportunity for deaf people to talk with vocal humans without the need of an interpreter. The system is built for the automated conversion of ASL into textual content and speech.

In our project we primarily focus on producing a model which can recognize Fingerspelling based hand gestures in order to form a complete word by combining each gesture. The gestures we aim to train are as given in the image below.

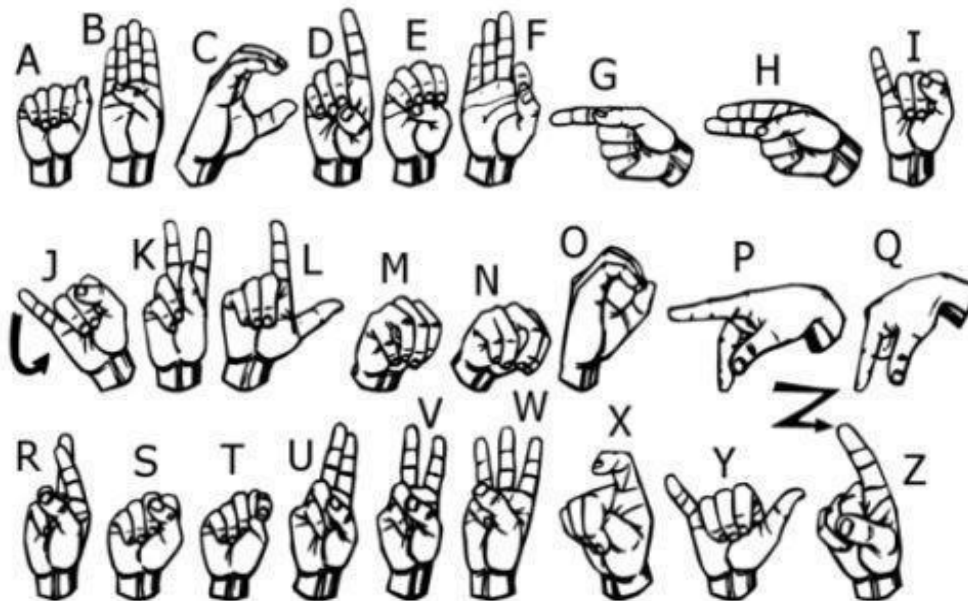


Figure 1: Hand Gestures

1.2 Background

Traditional methods of sign language recognition have relied on wearable sensors or bulky hardware . However, the rise of deep learning and computer vision techniques enables accurate, contactless recognition using just a webcam and software frameworks .

1.3 Problem Statement

There is a need for an accurate, real-time, and scalable solution to interpret sign language and convert it into readable formats without requiring physical contact or specialized hardware .

Chapter 2

Literature Review

2.1 Review of literature

Morillas-Espejo & Martínez-Martín, 2024 addresses the communication barriers faced by individuals with hearing impairments in Spain, where over 13% of the population is affected. The research focuses on Spanish Sign Language (Lengua de Signos Española), particularly on finger spelling, which remains a crucial yet under-supported mode of communication due to the lack of widespread societal understanding. To facilitate real-time sign language recognition and bridge this gap, the authors developed a novel image dataset covering all 30 letters of the Spanish Sign Language alphabet, including both static and dynamic signs. They conducted a comparative analysis using various deep learning models—Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Vision Transformers—for both spatial and temporal analysis. The models were trained and evaluated on the custom dataset, with the best model achieving an accuracy of 79.96% on previously unseen data. The study concludes with the implementation of a real-time platform that can interpret finger-spelled words in LSE, demonstrating its potential to support more inclusive communication systems.

Mizan et al., 2024 focuses on improving Sign Language Recognition (SLR) systems to aid communication for the deaf and hard-of-hearing. It compares three models—EfficientNetB3, Inception-v3, and GoogLeNet—using 27,000 ASL images across 27 classes. EfficientNetB3 achieved the highest test accuracy after fine-tuning. The model was then integrated into a real-time Android app, enhancing communication accessibility and promoting inclusivity.

Deshmukh et al., 2024 develops a system for real-time Marathi sign language recognition to aid communication for deaf and dumb individuals. It uses a custom dataset and evaluates deep learning models like VGG-16, which achieved high accuracy rates of 95.55% for validation and 97.38% for testing. A user-friendly GUI

was also created to map hand gestures to Marathi letters, enhancing communication with the general public .

Sakthivel et al., 2024 introduces a real-time sign language recognition system to aid communication for those with speech or hearing impairments. It uses the Mediapipe model and LSTM networks to interpret ASL gestures by analyzing key landmarks on the body, hands, and face. The system's CNN-LSTM architecture captures essential features, achieving 92% accuracy on a diverse ASL dataset. This approach is promising for assistive technologies and educational tools, providing an accessible solution for sign language interpretation .

Balachandra & Manjula, 2024 presents a real-time translation system for the hearing impaired, converting spoken language into text and sign language using deep learning. It features a speech-to-text model with 96.5% accuracy, a sign language recognition module with 92.3% accuracy, and a text-to-sign translation model with a BLEU score of 88.7. The system operates with a 150-millisecond latency, ensuring real-time use, and is effective in various environments. Future plans include supporting more languages and enhancing privacy with edge computing .

Shekhar, 2024 investigates converting American Sign Language (ASL) gestures into text using deep learning models and MediaPipe systems. It evaluates various models, with MobileNet achieving a validation accuracy of 95.6% and a test accuracy of 97.8% . The CNN model with MediaPipe outperformed others, with a validation accuracy of 98.5% and a test accuracy of 99.5%, making it the best for accurate ASL gesture detection . This technology could significantly improve sign language accessibility for the deaf and hard of hearing communities .

Aravind et al., 2024 introduces a method to improve American Sign Language (ASL) recognition using transfer learning with the MobileNetV2 architecture, achieving high accuracy with a large dataset of 274,612 RGB images . The model, trained on Kaggle's Dual GPU T4 infrastructure, uses categorical cross-entropy loss and the Stochastic Gradient Descent optimizer, resulting in 99.82% training accuracy and 99.14% validation accuracy . It processes in real-time at 99 milliseconds per step,

making it suitable for applications in education and assistive devices . Future research aims to explore dialect and gesture variations for enhanced accessibility .

Prianka et al., 2024 introduces a three-stream, multi-modal architecture with an API for sign language translation, integrating sentence correction, emotion detection, and gesture recognition models to enhance translation accuracy and emotional context . Preliminary results show improved translation accuracy and reduced semantic errors, benefiting the deaf and hard-of-hearing community by offering more inclusive communication technology .

Kabir et al., 2025 presented an advanced approach for Bangla Sign Language (BdSL) recognition using a Max Voting Ensemble technique that combines multiple pre-trained deep learning models, including Xception, InceptionV3, DenseNet121, ResNet50, and MobileNetV2. While prior research has focused on sign language recognition for English, Arabic, and Japanese, BdSL remains underexplored. The proposed ensemble method enhances classification accuracy by leveraging the complementary strengths of individual models, which were trained on BdSL-38 and BdSL-49 datasets. Through preprocessing and model integration, predictions are aggregated using majority voting. This technique achieved high test accuracies of 96.62% and 99.92%, demonstrating superior reliability over standalone models and offering a robust solution for BdSL recognition.

Myagila et al., 2025 explored the use of pose estimation features for sign language recognition, addressing challenges related to signer independence and hand dominance. The study proposed a two-stream model architecture incorporating Gated Recurrent Units (GRUs) with ELU activation to improve computational efficiency and learning performance. Using a Tanzania Sign Language dataset collected via mobile devices and pose features extracted through MediaPipe Holistic, the model achieved 95% overall accuracy. In signer-independent settings, it demonstrated strong performance, with a notable 37% improvement in accuracy for left-handed signers. The two-stream approach effectively addressed hand dominance variability, making it suitable for real-world applications with limited computational resources.

Guan et al., 2025 proposed a multi-stream keypoint attention network (MSKA-SLR) to enhance sign language recognition (SLR) and translation (SLT), addressing the limitations of traditional RGB-based methods which are sensitive to background variations and computationally intensive. By leveraging keypoint sequences generated through pose estimation, the model reduces background dependency and improves efficiency. Inspired by human cognition, the approach models the relationships between gesture configurations and contextual cues using multi-stream attention mechanisms, including keypoint fusion, head fusion, and self-distillation. Extensive evaluations on benchmarks such as Phoenix-2014, Phoenix-2014T, and CSL-Daily demonstrated the framework's effectiveness, achieving state-of-the-art performance on the Phoenix-2014T SLT task.

Wang et al., 2022 introduced DeepSLR, a real-time, end-to-end sign language recognition (SLR) system aimed at overcoming limitations in continuous recognition and accuracy, particularly in capturing both finger and arm movements. Unlike previous systems such as SignSpeaker, which struggle with two-handed signs, DeepSLR employs two armbands equipped with IMU and multi-channel sEMG sensors to effectively capture detailed motion data. The system utilizes an attention-based encoder-decoder architecture with multi-channel CNNs to eliminate the need for sign segmentation. Implemented on a smartphone, DeepSLR achieved a low average word error rate of 10.8% and demonstrated real-time performance, recognizing four-word sentences in under 1.1 seconds.

Sau et al., 2022 conducted a comprehensive literature survey on various methodologies and models used for Sign Language Recognition (SLR), emphasizing its role in enabling two-way communication between sign language users and non-users. While SLR has gained popularity in computer vision research, its practical implementation remains limited due to high resource requirements and the complexity of model development. The survey primarily focused on approaches applied to American Sign Language (ASL), highlighting both the promising results achieved and the persistent challenges in areas such as model generalizability and scalability. The study underscores the need for more efficient and inclusive SLR systems.

Raval & Gajjar, 2021 proposed a real-time sign language recognition system aimed at bridging communication gaps for individuals with speech impairments. The system

utilizes image processing for background extraction and pre-processing, followed by machine learning techniques for gesture recognition. A Convolutional Neural Network (CNN) is used to classify hand gestures representing the 24 alphabets of the English language. The system was tested on a custom-made dataset and real-time hand gestures performed by individuals of different skin tones, achieving an accuracy of 83%. This work contributes to enhancing communication for the differently-abled, particularly the deaf and mute communities.

Lee et al., 2021 developed an ASL learning application prototype incorporating real-time sign recognition using a Leap Motion Controller to enhance ASL education. The system, embedded in a Whack-a-Mole game format, uses both static and dynamic signs, employing a hybrid classification model combining Long Short-Term Memory (LSTM) networks with k-Nearest Neighbors (k-NN) to handle sequential input data. Key gesture features—such as finger angles, inter-finger distances, and sphere radius—were extracted for classification. Trained on 2,600 samples (100 per alphabet), the system achieved a high recognition accuracy of 99.44% and 91.82% in 5-fold cross-validation, demonstrating the potential of Leap Motion technology for effective ASL learning tools.

Muthu Mariappan & Gomathi, 2019 proposed a real-time sign language recognition system designed for Indian Sign Language (ISL), focusing on both hand gestures and facial expressions. The system uses OpenCV-based skin segmentation to identify and track Regions of Interest (ROI), and applies the Fuzzy C-Means clustering algorithm for gesture training and prediction. Applications of the system include gesture-controlled robotics, smart environments, gaming, Human-Computer Interaction (HCI), and sign language interpretation. This approach provides an accessible communication tool for individuals with hearing and speech impairments, enhancing their interaction with the broader community.

Taskiran et al., 2018 developed a real-time sign language recognition system aimed at enabling communication between individuals unfamiliar with sign language and the hearing-impaired community. Focusing on American Sign Language (ASL), the system employs a Convolutional Neural Network (CNN) trained on a dataset collected in 2011 by Massey University, achieving 100% test accuracy. For real-time implementation, skin color detection is used to identify the hand region, and gestures

are recognized using the convex hull algorithm. The trained model and its weights are then used to classify gestures in real time, with an achieved accuracy of 98.05%.

Likhar et al., 2020 proposed a dual-approach system to recognize Indian Sign Language (ISL) and bridge the communication gap between specially-abled individuals and the general population. The first approach utilized RGB-D data captured via Microsoft Kinect, applying 3D reconstruction and affine transformation to segment hand regions, which were then classified using CNNs. This system recognized 36 static ISL gestures with 98.81% accuracy and achieved 97.71% on American Sign Language (ASL) via transfer learning. Additionally, 10 dynamic gestures were trained using a convolutional LSTM, yielding 99.08% accuracy. However, due to Kinect's bulk and computational demands, a second approach employed U-Net with ResNet-101 for multi-class semantic segmentation using standard RGB input. This method achieved high performance, with an IoU of 0.9920 and F1-score of 0.9957, making it suitable for real-time applications.

Gupta et al., 2024 developed a sign language recognition model aimed at simplifying communication between deaf and mute individuals and the general public. The system utilizes deep learning techniques, incorporating two prominent convolutional neural networks—ResNet50 and VGG16. These networks were trained on a globally diverse linguistic dataset, enabling real-time gesture recognition and image-to-text conversion. The application was designed to be user-friendly and accessible online, with potential applications in both the educational and healthcare sectors, allowing deaf and mute users to access information and communicate effectively with educators or medical professionals. The model trained on a dataset split with 70% for training and 30% for testing, demonstrating high accuracy in gesture recognition across both static and real-time images.

Koonce (2021) proposed the use of ResNet-50 as a foundational deep learning architecture, emphasizing its role as a benchmark in computer vision research. The study highlights that numerous sign language recognition (SLR) models are often evaluated against a ResNet-50 baseline, owing to its proven robustness and high performance. The availability of pre-trained weights on large-scale datasets such as ImageNet makes ResNet-50 highly suitable for transfer learning. By modifying the final layers of the pre-trained network, researchers can effectively retrain the model on

smaller, domain-specific datasets. Instead of designing new architectures from scratch, leveraging ResNet-50—alongside techniques such as data augmentation—has been shown to enhance performance and reduce training time. Therefore, it is widely recommended as a starting point for real-world image recognition tasks, including those in the SLR domain.

Dong et al. (2020) investigated the performance of the MobileNetV2 model in the domain of image classification, a core task in computer vision where machine learning has seen widespread global adoption. In their study, MobileNetV1 was used as a baseline model to comparatively evaluate the enhancements introduced in MobileNetV2. The authors conducted experiments using various image datasets sourced from TensorFlow, and employed the T-SNE (t-distributed Stochastic Neighbor Embedding) technique for visualizing the model's feature representations. Their comparative analysis revealed that MobileNetV2 outperformed its predecessor not only in classification accuracy but also in computational efficiency. Furthermore, they conducted extensive optimization experiments to validate the effectiveness of MobileNetV2, thereby reinforcing its suitability for lightweight and high-accuracy computer vision applications.

2.2 Research Gaps/ Motivation of Work

2.2.1 Research Gaps

Despite notable advancements in sign language recognition systems, several critical research gaps remain that present opportunities for further exploration and enhancement. One key area is the robustness of recognition models in real-world conditions. Existing models often experience performance degradation under varying lighting, backgrounds, or camera quality, which limits their practical applicability in uncontrolled environments. Enhancing model stability and generalization across diverse conditions remains a significant challenge.

Another gap lies in the limited scope of existing datasets. Current datasets, although substantial in size (e.g., 27,000 images across 27 classes), often lack the diversity and complexity required to effectively recognize a wider variety of gestures, including those that are less common or more nuanced. Expanding the dataset to include additional signs and variations can improve the comprehensiveness and adaptability of recognition systems.

Moreover, the real-time performance of such systems requires improvement. High processing time and latency can negatively impact user interaction, particularly in dynamic scenarios where immediate feedback is essential. Optimizing models for faster and more efficient execution is thus a crucial area of focus.

The user interface design also presents an opportunity for development. Although current implementations may be user-friendly, there is potential to enhance accessibility and ease of use, especially for individuals with different levels of technological proficiency. Incorporating user-centered design principles can help in creating more intuitive and inclusive interfaces.

2.2.2 Motivation of work

For interaction between normal people and Deaf & Mute (D&M) individuals, a language barrier exists due to the difference in the structure of sign language compared to normal text. As a result, D&M individuals depend on vision-based communication to interact with others. If there were a common interface that could convert sign language into text, it would enable non-D&M people to easily understand the gestures. This has led to significant research aimed at developing a vision-based interface system that allows D&M individuals to communicate without needing to understand each other's language.

The goal is to create a user-friendly Human-Computer Interface (HCI) that allows computers to understand human sign language. There are various sign languages around the world, such as American Sign Language (ASL), French Sign Language, British Sign Language (BSL), Indian Sign Language, and Japanese Sign Language. Research has been conducted on these and other sign languages globally, contributing to the development of more effective communication systems for D&M individuals.

Chapter 3

Present Work

3.1 Formulation of problem

Real time Sign language recognition (SLR) systems play a crucial role in assisting communication between the deaf and hard-of-hearing community and the general population. With growing interest in inclusive technologies, these systems aim to translate visual gestures into spoken or written language. Although research in this area has made significant progress, especially with the use of machine learning and computer vision techniques, current systems still face substantial challenges when it comes to real-world application. Most SLR models perform well in controlled environments but struggle in dynamic conditions. Everyday variations such as changing lighting, complex or cluttered backgrounds, and inconsistent camera positions can lead to decreased recognition accuracy. These environmental challenges make it difficult for current systems to deliver consistent results in practical use, which limits their adoption and effectiveness.

Another key limitation is the narrow scope of training data. Many available datasets include only a small number of signs and are collected in clean environments using a limited number of signers. This restricts the system's ability to generalize to new users, diverse signing styles, or regional variations in sign language. Furthermore, most systems are trained on isolated gestures rather than continuous signing, which is how communication naturally occurs. This makes it difficult for the model to understand the context or recognize signs in a flowing sequence, as would be expected in a real conversation. The lack of extensive, varied, and realistic datasets poses a serious challenge to the development of SLR systems that can perform well across different real-world scenarios.

Real-time performance is another important factor that often goes under-addressed. High computational requirements can result in significant delays between gesture input and system response, interrupting the natural flow of communication. To be practical and user-friendly, an SLR system must provide fast and accurate recognition without requiring high-end hardware. Many existing models achieve accuracy but compromise on speed, or vice versa. This trade-off creates a barrier for real-time

applications where both speed and precision are equally critical. Additionally, user interface design is often neglected. Tools that are too technical or difficult to navigate can discourage the very users they are meant to help. A well-designed, simple, and intuitive interface is essential to ensure that the system is accessible to users of all backgrounds and skill levels.

Lastly, while many SLR systems explore complex hardware setups involving sensors or specialized equipment, this research focuses on a more practical and widely accessible solution—relying solely on input from a standard RGB camera. This approach not only reduces cost and complexity but also allows the system to be easily used on commonly available devices such as laptops or webcams. However, this also presents a challenge: achieving high accuracy and real-time performance without the aid of extra sensors requires efficient model design and careful preprocessing. The aim of this research is to develop a robust, real-time sign language recognition system that works effectively in varied environments using only basic visual input. By overcoming current limitations in environmental adaptability, data scope, speed, and usability, this system can contribute meaningfully to more inclusive and accessible communication tools for the deaf and hard-of-hearing community.

3.2 Objectives.

1. To integrate two labeled sign language datasets for training deep learning models.
2. To design and implement a real-time system that captures and processes hand gestures for accurate sign language recognition.
3. To evaluate the performance of the proposed system using evaluation metrics such as precision, recall, F1-score, and the confusion matrix.

3.3 Methodology

The methodology for this project focuses on developing a real-time sign language recognition system using deep learning and computer vision techniques. The system is designed to detect and classify static hand gestures representing alphabets and digits using a live webcam feed. To achieve accurate and efficient recognition, two powerful Convolutional Neural Network (CNN) architectures—ResNet50 and MobileNetV2—were employed. These models were selected for their high performance in image classification tasks and their ability to generalize well across varied input conditions. The entire process, from dataset creation and preprocessing to model training, testing, and real-time deployment, is discussed in the subsequent sections. OpenCV was used for real-time video capture and gesture tracking, while TensorFlow and Keras facilitated model development and integration.

The proposed system is designed to recognize static hand gestures from live video input and convert them into readable text and speech. It operates through multiple stages, starting with real-time video capture followed by hand detection and preprocessing. The processed frames are then fed into a deep learning model for gesture classification. Once classified, the system maps the gesture to its corresponding text. This text is displayed on the screen and also converted into speech. The approach enables seamless communication for individuals with hearing or speech impairments. The step-by-step framework is discussed below:

3.3.1 Data Collection

For this project, two datasets were sourced from Kaggle: one containing images of hand gestures representing digits (0) in American Sign Language (ASL) and the other consisting of alphabets (A-B). These datasets were initially available in various formats, and to ensure consistency and compatibility with the model, both were converted to JPG format.

The datasets were then combined into a single dataset, with each image clearly labeled according to the corresponding gesture. Images in the dataset were captured under varying conditions, including different hand shapes, background variations, and lighting to ensure robustness in recognizing gestures across real-time scenarios.

Each image was stored with a label representing the corresponding sign language gesture (e.g., 'A', 'B'), and the images were organized into directories based on their class labels for easy access during model training.

3.3.2 Preprocessing Steps

Preprocessing plays a crucial role in enhancing the quality of input data, improving the model's performance, and ensuring consistency across the dataset. The following preprocessing techniques were applied to the sign language gesture images before they were fed into the deep learning models:

Grayscale Conversion:

All images were converted to grayscale to simplify the input data and reduce the complexity of the model. This helps focus the model's attention on shape and gesture recognition rather than color variations, which are irrelevant to hand gesture recognition.

Gamma Correction:

To adjust the brightness and contrast of the images, gamma correction was applied. This technique enhances visibility by compensating for lighting inconsistencies, ensuring that the model learns features from images captured under various lighting conditions.

CLAHE (Contrast Limited Adaptive Histogram Equalization):

CLAHE was applied to further improve the contrast of the images. This method adapts the enhancement based on the local region of the image, making the model more effective in detecting fine details in hand gestures under different lighting.

Gaussian Blur:

A Gaussian blur was applied to reduce noise and smooth the image, helping to eliminate unnecessary pixel variations and allowing the model to focus on the essential features of the hand gestures.

Resizing:

All images were resized to a consistent size of 224x224 pixels, which is the input size required by the models (ResNet50 and MobileNetV2). Resizing ensures that the

images are compatible with the neural network architecture while maintaining the gesture's structural integrity.

Normalization:

Pixel values were normalized by scaling them to a range between 0 and 1. This step helps speed up the training process by ensuring that the input values are within a consistent range and improves convergence during model training.

3.3.3 Model Selection

For the sign language recognition task, two deep learning models—ResNet50 and MobileNetV2—were selected based on their proven performance in image classification tasks and their ability to handle real-time predictions efficiently.

ResNet-50 is a type of convolutional neural network (CNN) that has revolutionized the way we approach deep learning. It was first introduced in 2015 by Kaiming He et al. at Microsoft Research Asia.

ResNet stands for residual network, which refers to the residual blocks that make up the architecture of the network. ResNet-50 is based on a deep residual learning framework that allows for the training of very deep networks with hundreds of layers.

The ResNet architecture was developed in response to a surprising observation in deep learning research: adding more layers to a neural network was not always improving the results. This was unexpected because adding a layer to a network should allow it to learn at least what the previous network learned, plus additional information.

To address this issue, the ResNet team, led by Kaiming He, developed a novel architecture that incorporated skip connections.

These connections allowed the preservation of information from earlier layers, which helped the network learn better representations of the input data. With the ResNet architecture, they were able to train networks with as many as 152 layers.

The results of ResNet were groundbreaking, achieving a 3.57% error rate on the ImageNet dataset and taking first place in several other competitions, including the ILSVRC and COCO object detection challenges.

This demonstrated the power and potential of the ResNet architecture in deep learning research and applications. the basic architecture of the ResNet-50 network.

ResNet-50 Architecture

ResNet-50 consists of 50 layers that are divided into 5 blocks, each containing a set of residual blocks. The residual blocks allow for the preservation of information from earlier layers, which helps the network to learn better representations of the input data.

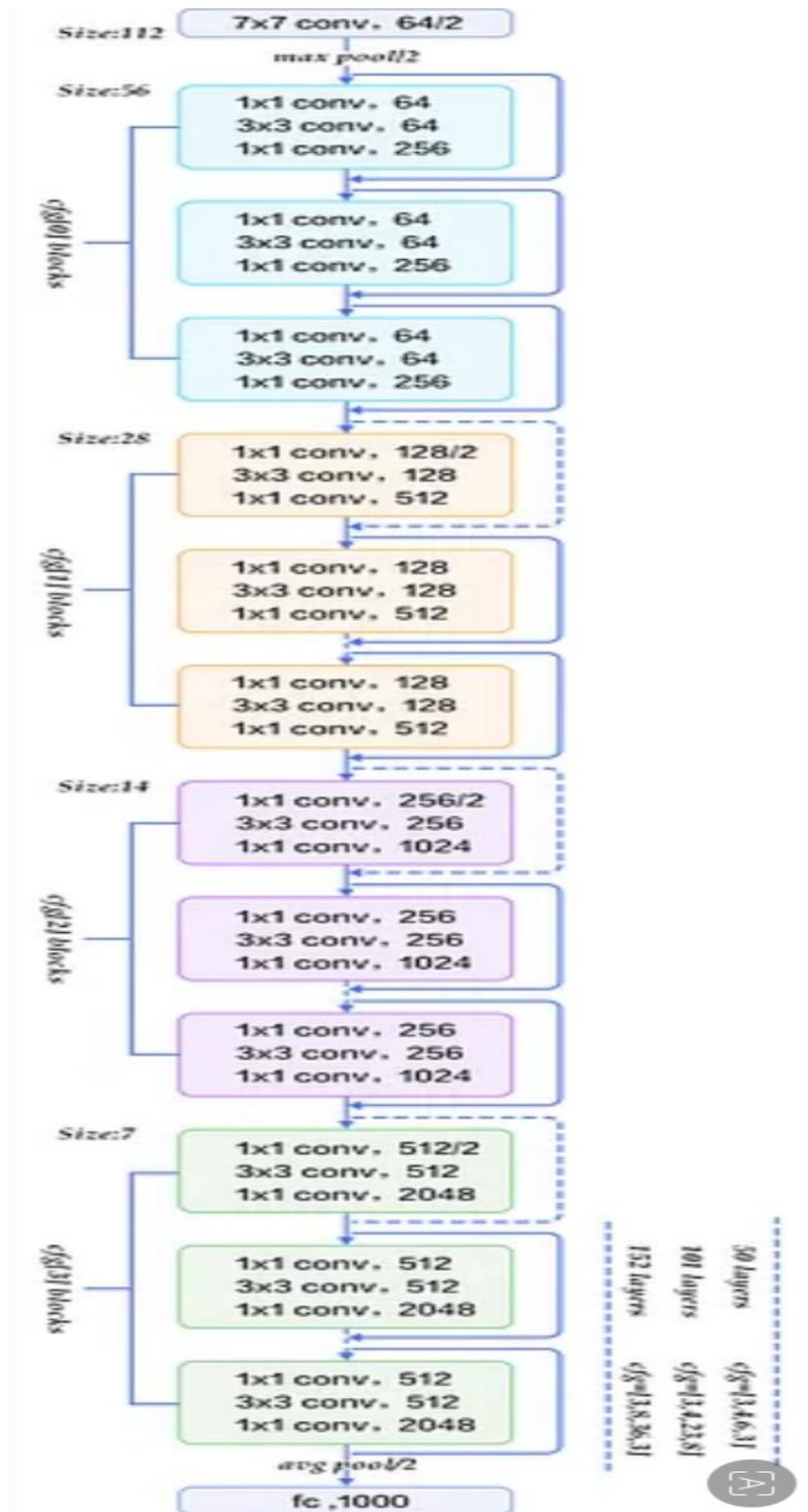


Figure 2: ResNet-50Architecture

The following are the main components of ResNet50.

1. Convolutional Layers

The first layer of the network is a convolutional layer that performs convolution on the input image. This is followed by a max-pooling layer that downsamples the output of the convolutional layer. The output of the max-pooling layer is then passed through a series of residual blocks.

2. Residual Blocks

Each residual block consists of two convolutional layers, each followed by a batch normalization layer and a rectified linear unit (ReLU) activation function. The output of the second convolutional layer is then added to the input of the residual block, which is then passed through another ReLU activation function. The output of the residual block is then passed on to the next block.

3. Fully Connected Layer

The final layer of the network is a fully connected layer that takes the output of the last residual block and maps it to the output classes. The number of neurons in the fully connected layer is equal to the number of output classes.

Concept of Skip Connection

Skip connections, also known as identity connections, are a key feature of ResNet-50. They allow for the preservation of information from earlier layers, which helps the network to learn better representations of the input data. Skip connections are implemented by adding the output of an earlier layer to the output of a later layer.

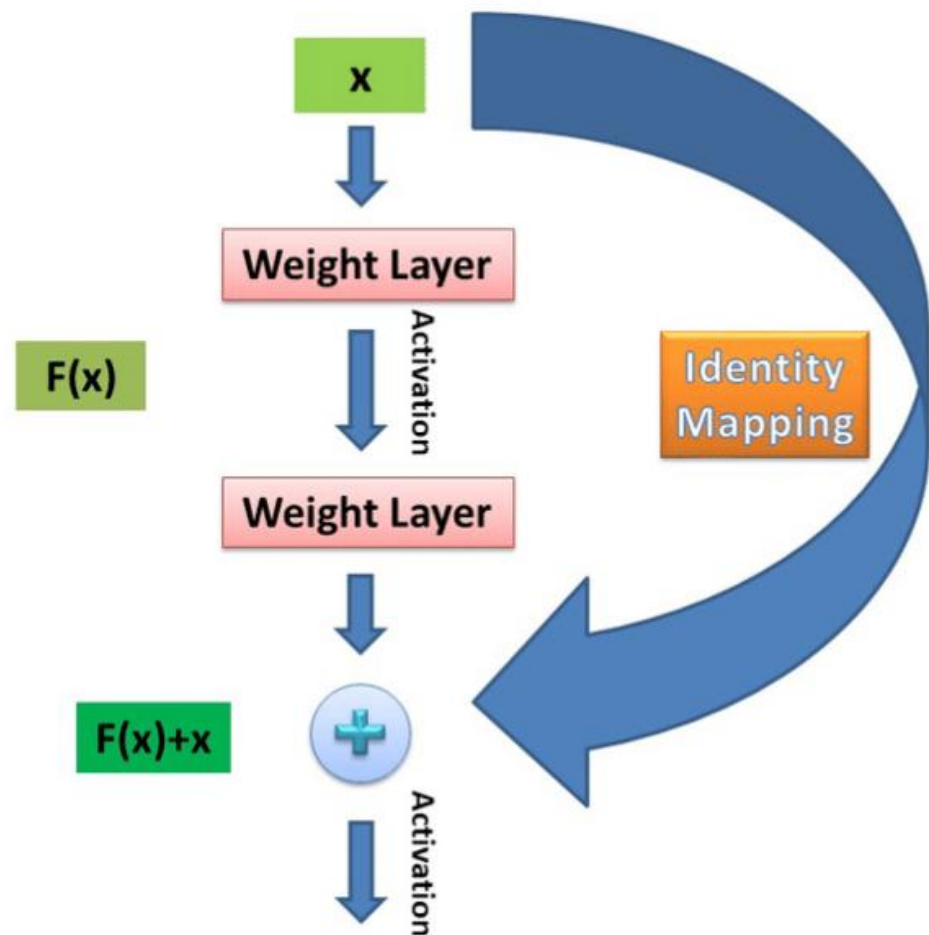


Figure 3: Skip Connection

A lightweight convolutional neural network (CNN) architecture, MobileNetV2, is specifically designed for mobile and embedded vision applications. Google researchers developed it as an enhancement over the original MobileNet model. Another remarkable aspect of this model is its ability to strike a good balance between model size and accuracy, rendering it ideal for resource-constrained devices.

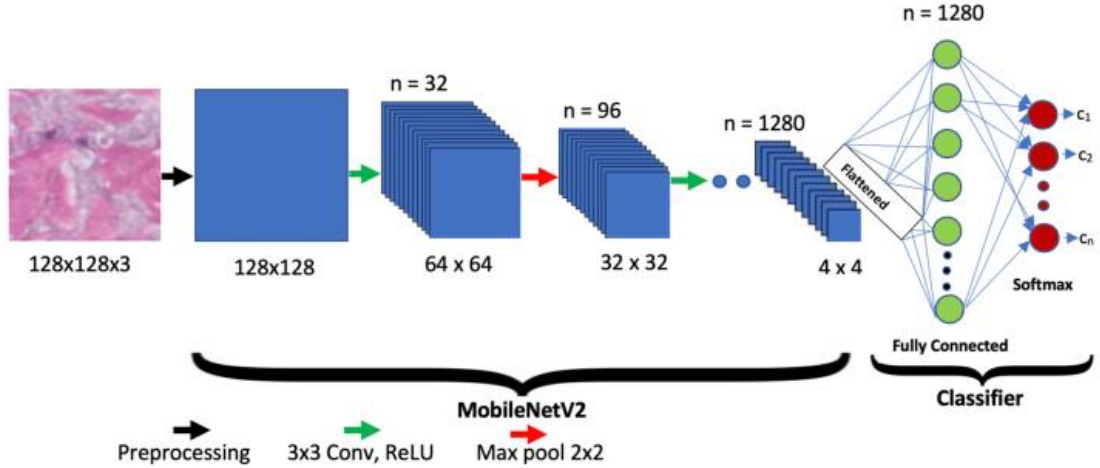


Figure 4: mobileNetv2

Key Features

Mobilenetv2 architecture incorporates several key features that contribute to its efficiency and effectiveness in image classification tasks. These features include depthwise separable convolution, inverted residuals, bottleneck design, linear bottlenecks, and squeeze-and-excitation (SE) blocks. Each of these features plays a crucial role in reducing the computational complexity of the model while maintaining high accuracy.

3.4 Training the Model

The model training phase was carried out using the ResNet50 architecture to classify various sign language gestures. This step played a critical role in enabling the system to accurately interpret hand gestures in real-time.

3.4.1 Dataset Preparation

The dataset used for training was a combination of two publicly available sign language datasets sourced from Kaggle. These images were organized into structured directories for training and validation, with separate folders for each class label. All images were resized to 224x224 pixels to match the input requirements of the ResNet50 model. The data was normalized by scaling pixel values to fall within the range [0, 1].

3.4.2 Data Augmentation

To enhance the model's ability to generalize to unseen data and reduce the risk of overfitting, data augmentation techniques were applied during the training process. These techniques included rotation, zooming, horizontal flipping, and shifting of images. The validation dataset, however, was only rescaled without any augmentation to maintain consistency for performance evaluation.

3.5 Model Training and Monitoring

Training was performed on the preprocessed and augmented dataset. Throughout the training process, both training and validation accuracies were monitored to evaluate the model's performance. The model was saved after completion for future inference tasks. Accuracy trends were visualized using performance graphs to assess learning behavior and convergence across epochs.

3.6 Real-Time Sign Language to Text and Speech System

Following the successful training of the sign language recognition model using MobileNetV2, a real-time system was implemented to demonstrate the practical utility of the model. This system captures hand gestures from a live webcam feed, classifies them using the trained deep learning model, and converts the output into both textual and spoken forms.

3.6.1 System Overview

The real-time application integrates computer vision, deep learning, and text-to-speech technologies to enable seamless human-computer interaction for users communicating through sign language. It supports gesture recognition and speech output, offering a complete assistive communication system.

3.6.2 Functional Workflow

The system works as follows:

Live Video Feed: A webcam captures real-time video frames.

Region of Interest (ROI): A fixed area within the frame is defined for capturing hand gestures, ensuring consistent input.

Preprocessing: The ROI is resized to 224×224 pixels, normalized, and reshaped to match the model's input format.

Prediction: The preprocessed image is passed to the trained MobileNetV2 model which predicts the corresponding gesture with a confidence score.

Stability Check: To ensure reliable recognition, a gesture is only considered valid if it is consistently predicted over a certain number of frames (e.g., 15).

Text Generation: Recognized gestures are appended to a sentence string.

Speech Output: The recognized gesture is spoken aloud using a text-to-speech (TTS) engine (pyttsx3).

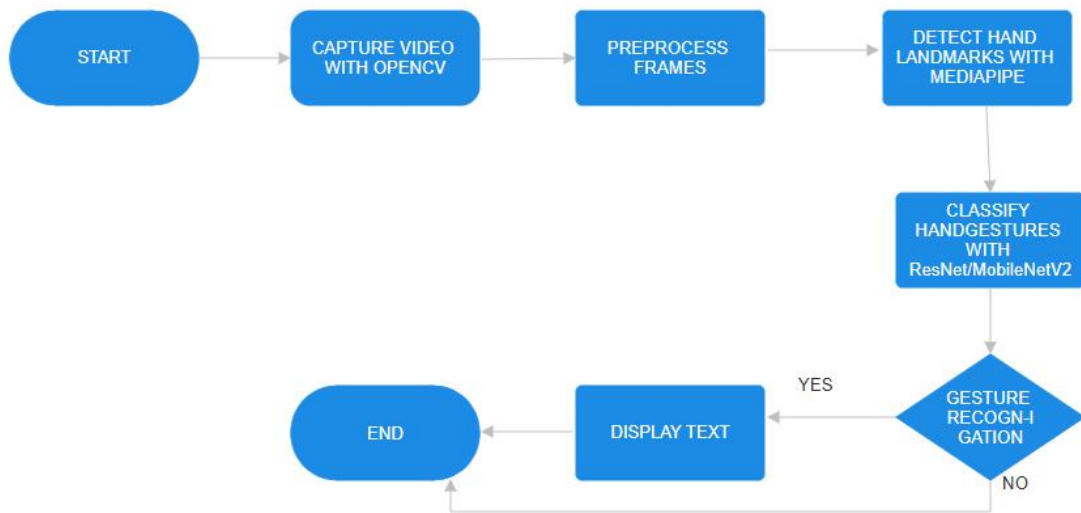


Figure 5: Real-Time Working of the Sign Language Recognition System

3.4 Description of Platform Used

This section details the hardware and software components utilized for the development and execution of this project. The selection of these components was driven by the need to efficiently process image and video data, train machine learning models, and deploy the final solution.

3.4.1 Hardware Requirements

The following hardware specifications were considered essential for the project:

Standard Webcam: A standard webcam was required as the primary device for capturing video input. This device provides the necessary visual data for the application.

Processor: Intel i5 or above: An Intel Core i5 processor (or equivalent) was specified to ensure sufficient processing power for handling real-time video streams, running computer vision algorithms, and managing the overall application workflow. Higher-end processors in the i5 range or those in the i7 or i9 series would offer improved performance.

RAM: 8GB or above: A minimum of 8GB of RAM was deemed necessary to accommodate the memory demands of video processing, deep learning model execution, and concurrent program operations. More RAM (16GB or higher) would enhance the system's ability to handle larger datasets and complex models, leading to smoother performance.

GPU for faster model training (e.g., NVIDIA CUDA-compatible): A dedicated Graphics Processing Unit (GPU), particularly an NVIDIA CUDA-compatible one, was highly recommended. GPUs are well-suited for parallel processing, which is crucial for accelerating the training of deep learning models. While the application can function without a dedicated GPU, training times would be significantly longer.

4.1.2 Software Requirements

The project relied on the following software components:

Programming Language: Python: Python was chosen as the primary programming language due to its rich ecosystem of libraries and frameworks for computer vision and machine learning. Its versatility, readability, and extensive community support made it an ideal choice for this project.

Libraries: OpenCV, TensorFlow/Keras

OpenCV (Open Source Computer Vision Library): OpenCV is a comprehensive library of functions for computer vision, image processing, and video analysis. It was used for tasks such as capturing video from the webcam, processing frames, and displaying output.

TensorFlow/Keras: TensorFlow is an open-source machine learning platform. Keras, which can run on top of TensorFlow, provides a high-level API for building and training neural networks. These libraries were used to construct and train the core machine learning models.

Frameworks: TensorFlow Lite : TensorFlow Lite is a set of tools that enables on-device machine learning by optimizing TensorFlow models for mobile, embedded, and IoT devices. It was considered for optimizing the trained models to ensure efficient deployment and performance on less powerful hardware.

IDE: VS Code

VS Code (Visual Studio Code): VS Code is a popular source code editor developed by Microsoft. It provides a streamlined environment for coding, debugging, and version control. VS Code could have been used as a local IDE for writing the Python code.

Chapter 4

Results and Conclusions

4.1 Results

The performance of the proposed MobileNetV2-based model was evaluated by analyzing its training and validation accuracy across four epochs, as illustrated in the graph titled "Model Accuracy over Epochs." The training accuracy shows a consistent upward trend, beginning at approximately 95.67% in the first epoch and reaching 99.63% by the fourth epoch. Similarly, the validation accuracy starts high at around 99.80% and improves slightly to approximately 99.87% by the final epoch. This close alignment between training and validation accuracy throughout the training process indicates that the MobileNetV2 model is effectively learning the patterns from the data and generalizing well to unseen data. The steady improvement in both training and validation accuracy across epochs highlights the stability and robustness of the model. The results demonstrate that the chosen MobileNetV2 architecture and training approach are successful in achieving high predictive performance within a limited number of training epochs, confirming the model's efficiency and effectiveness for the intended classification task.

The performance of the proposed ResNet50-based model was evaluated over two epochs by analyzing its training and validation accuracy, as shown in the graph titled "ResNet50 Model Accuracy." The training accuracy began at approximately 37% in the first epoch and improved to around 57% by the second epoch, demonstrating a clear upward trend in learning. Meanwhile, the validation accuracy started at approximately 78% and increased to about 87%, indicating that the model was able to generalize well to the unseen data even in the early stages of training. Although the training accuracy is comparatively lower than the validation accuracy, the consistent improvement in both metrics across epochs reflects that the model is progressively learning meaningful representations from the data. The results suggest that while ResNet50 shows promise with notable gains in accuracy within just two epochs, further training could potentially enhance its performance significantly. The ResNet50 architecture demonstrates an ability to capture important features early in the training process, making it a strong candidate for deeper training and fine-tuning in future

work.

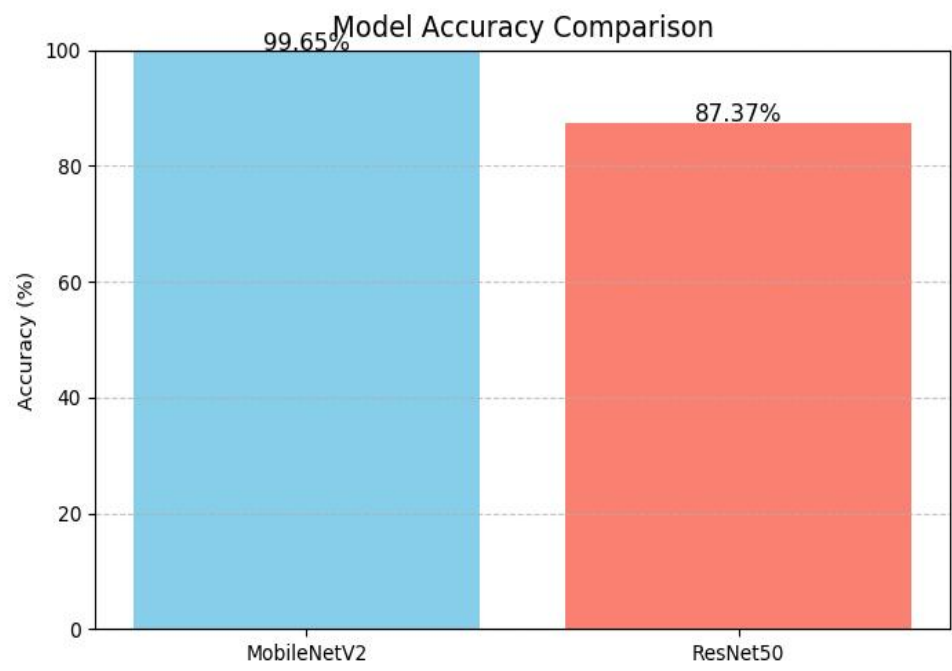


Figure 6:Model accuray comparison

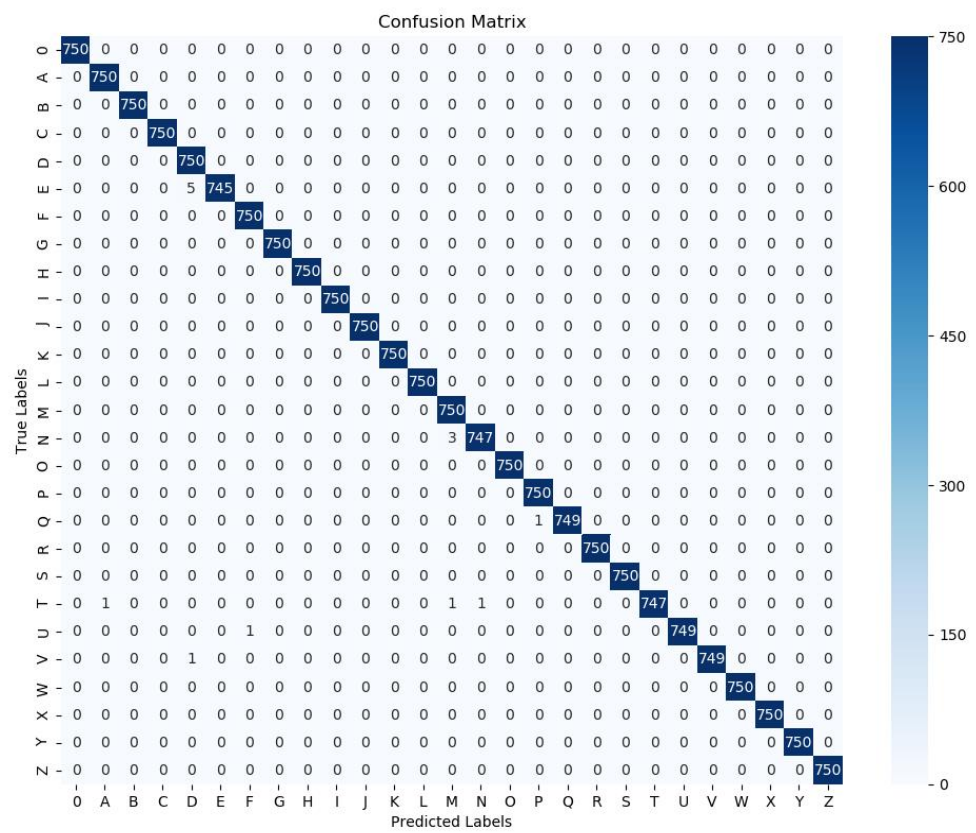


Figure 7:Confusion matric mobileNetV2

The displayed confusion matrix represents the performance of a multi-class classification model for 36 classes (0–9 and A–Z), showing exceptionally high accuracy. Each class contains 750 test samples, and the strong diagonal dominance indicates that the model correctly classified nearly all samples, with only a few minor misclassifications. For example, class 'F' was misclassified as 'E' five times, and class 'N' was predicted as 'M' three times, suggesting confusion between visually similar gestures. Most classes achieved perfect prediction, reflecting the model's robustness and reliability. Overall, the matrix demonstrates a highly accurate and well-trained model suitable for real time sign language recognition

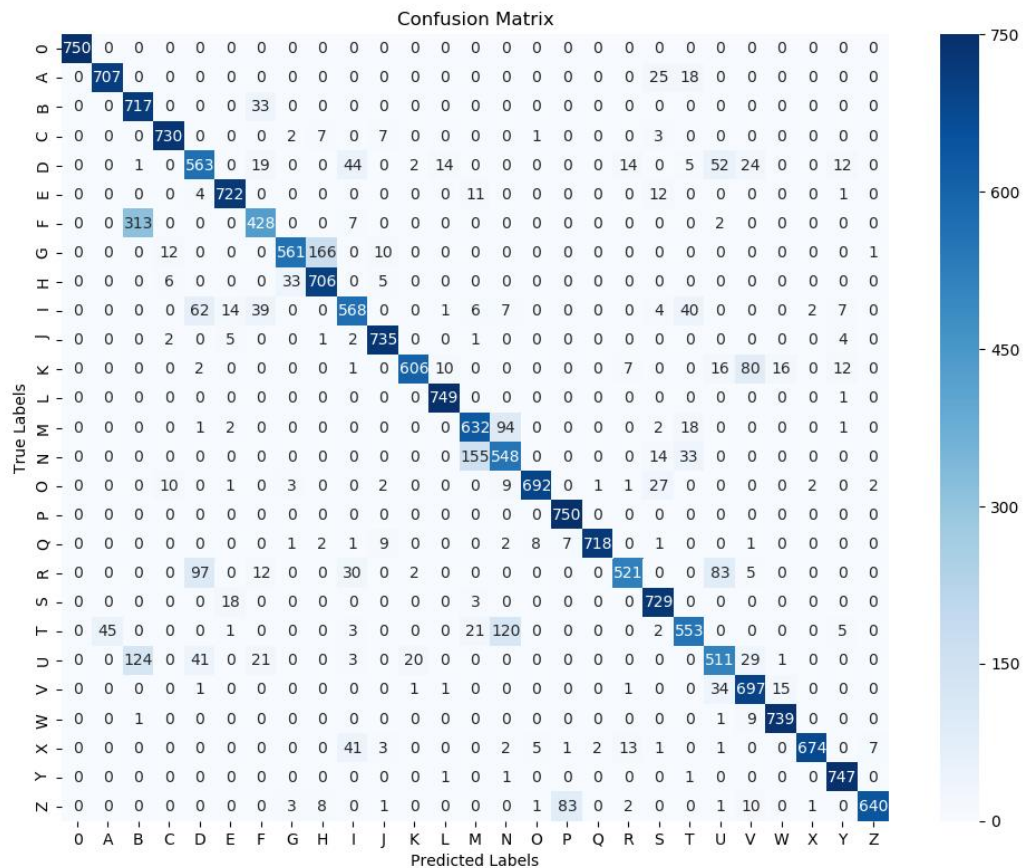


Figure 8:Confusion matrix ResNet50

This confusion matrix represents the performance of the ResNet50 model on a 36-class classification task involving letters (A–Z). Compared to a perfect confusion matrix, it shows significant misclassifications across several classes. While some

classes like ‘0’, ‘Q’, and ‘S’ are predicted with high accuracy, others—such as ‘G’, ‘H’, ‘M’, ‘N’, and ‘R’—exhibit considerable confusion with visually similar classes. For example, ‘G’ was correctly classified only 428 times out of 750 and misclassified as ‘F’, ‘E’, and ‘H’, while ‘R’ had frequent confusion with ‘B’, ‘D’, and ‘P’. These patterns indicate that although ResNet50 performs moderately well, it struggles with distinguishing between certain similar gestures, highlighting the need for improved feature learning or better dataset augmentation.

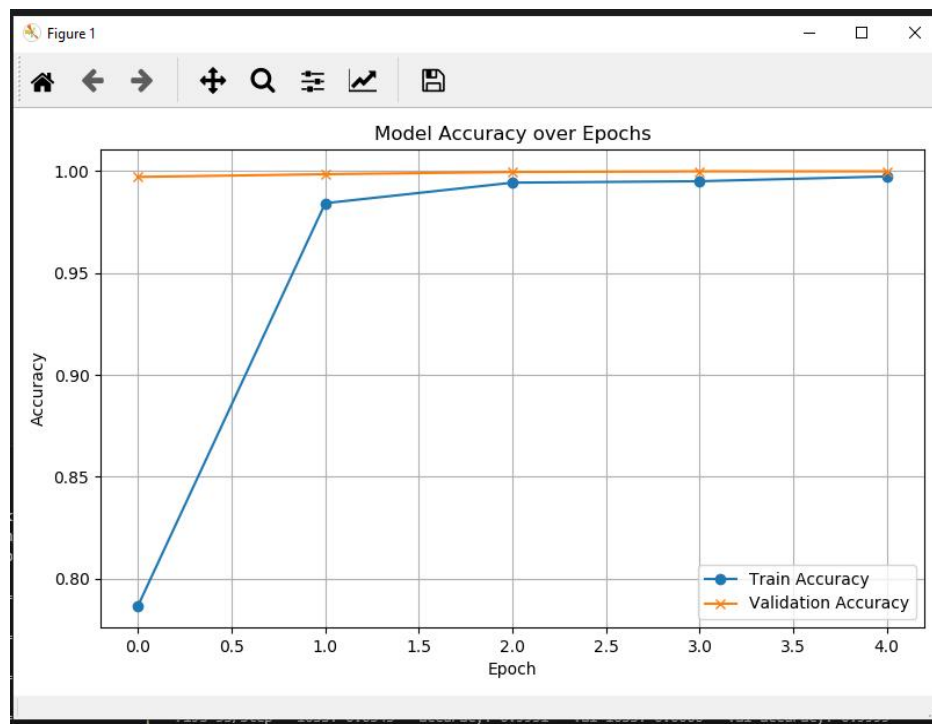


Figure 9: MobileNetV2 Model accuracy

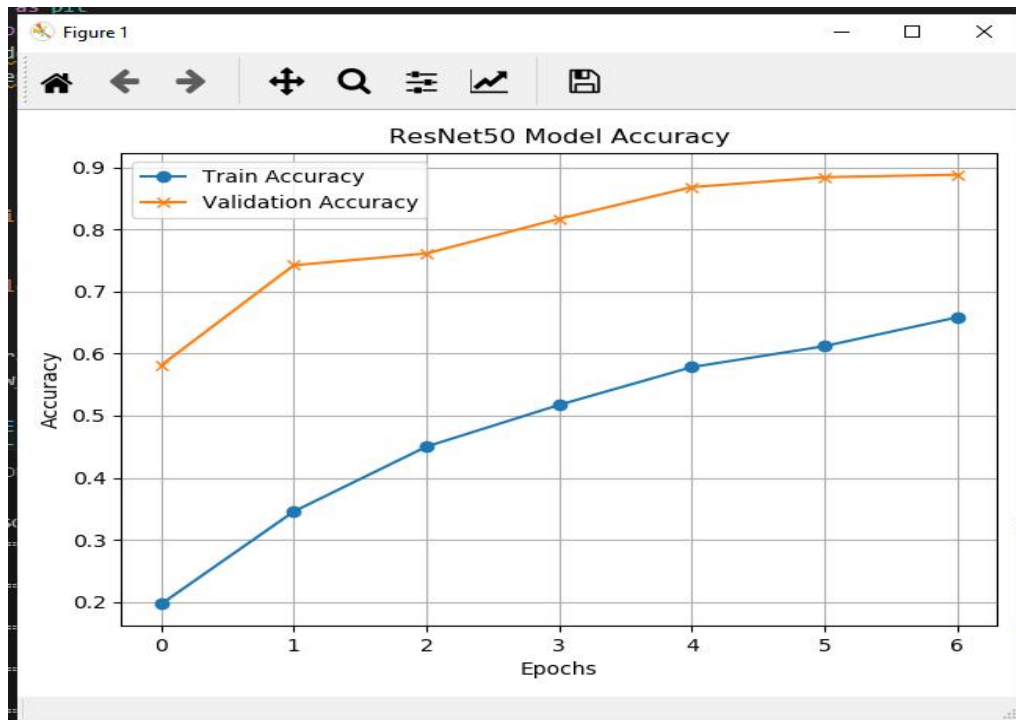


Figure 10: ResNet50 Model accuracy

Table 2 : Comparative Performance Analysis of MobileNetV2 and ResNet-50

Performance Metric	MobileNetV2 (%)	ResNet-50 (%)
Precision	99.86	89.81
Recall	99.94	89.86
F1-Score	99.90	89.71

OUTPU

Sign Language to Sentence (Two Model Comparison)

Upload hand gesture images and see predictions from both MobileNet and ResNet models

 Upload Multiple Hand Gesture Images



Drag and drop files here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



5.jpg 9.7KB



4.jpg 9.6KB



3.jpg 8.6KB



Showing page 1 of 2



Uploaded Images & Model Predictions

 1.jpg



 2.jpg



 3.jpg



 4.jpg



 5.jpg



Final Sentences from Both Models

MobileNetV2: GNDEC

ResNet: GNDEC



Sign Language to Sentence (Two Model Comparison)

Upload hand gesture images and see predictions from both MobileNet and ResNet models

Upload Multiple Hand Gesture Images



Drag and drop files here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



7.jpg 10.6KB

×



6.jpg 10.2KB

×



5.jpg 7.2KB

×

Showing page 1 of 1

< >



Uploaded Images & Model Predictions

1.jpg

▼

2.jpg

▼

3.jpg

▼

4.jpg

▼

5.jpg

▼

6.jpg

▼

7.jpg

▼



Final Sentences from Both Models

MobileNetV2: MACHINE

ResNet: MACHINE

4.2 Conclusions

This project aimed to develop a real-time sign language recognition system using deep learning and computer vision. The system is designed to assist hearing- and speech-impaired individuals by converting hand gestures into readable text and audible speech. Two powerful convolutional neural network models — MobileNetV2 and ResNet50 — were implemented and compared to evaluate performance in terms of accuracy and efficiency.

After preprocessing and augmenting the combined dataset of static hand gestures, both models were trained and tested. The MobileNetV2 model outperformed ResNet50, achieving a remarkable accuracy of 99.87%, demonstrating its suitability for real-time applications due to its lightweight architecture and fast inference time. In contrast, ResNet50 achieved an accuracy of 87.37%, still performing well but being more computationally intensive.

The final system includes live video feed capture using OpenCV, region-of-interest processing, gesture prediction, and real-time audio feedback using a text-to-speech engine. The real-time implementation proved to be stable, efficient, and user-friendly.

In conclusion, the project successfully meets its objective of providing an effective and practical solution for sign language recognition. The high accuracy of MobileNetV2, combined with its speed, makes it a strong candidate for real-world deployment on low-resource devices such as mobile phones or embedded systems.

References

- Morillas-Espejo, F., & Martinez-Martin, E. (2024). *A real-time platform for Spanish Sign Language interpretation. Neural Computing and Applications*. <https://doi.org/10.1007/s00521-024-10776-0>
- Mizan, Z. I., Aidelkhani, F., Reza, K. T., Bhuiyan, T. H., Alom, M., & Tuhin, S. I. (2024). *Real-Time Sign Language Recognition using Machine Learning and CNN-based EfficientNetB3 for Accessible Communication*. <https://doi.org/10.1109/icacrs62842.2024.10841529>
- Deshmukh, R., Lahamge, T., Phadatare, I., Shinde, D., & Manhas, R. (2024). *Real-Time Marathi Sign Language Recognition using Deep Learning Techniques*. <https://doi.org/10.1109/icacrs62842.2024.10841608>
- Sakthivel, S. M., Kumar, N. S., Harika, C., Harika, C., Reddy, C., & Kalyan, D. (2024). *Sign to Text: Automated Sign Language Interpretation using LSTM and Computer Vision*. <https://doi.org/10.1109/icacrs62842.2024.10841493>
- Balachandra, M., & Manjula, A. (2024). Multimodal Real-Time Translation System for Hearing Impaired Accessibility Using Deep Learning. *Indian Scientific Journal Of Research In Engineering And Management*. <https://doi.org/10.55041/ijsrem39280>
- Shekhar, R. (2024). *Real-time Sign Language to Text Conversion using Deep Learning Models*. <https://doi.org/10.1109/iconat61936.2024.10774717>
- Aravind, J. S., Reshma, R., Jaseela, B., Deepak, V. K., Lisha, G. P., & Shihabudeen, H. (2024). *Real-Time American Sign Language Gesture Recognition Using Transfer Learning*. <https://doi.org/10.1109/iccpct61902.2024.10672721>
- Prianka, R. R., Singh, T., Singh, R., Jaiswal, D., & Mathi, R. (2024). *A Comprehensive Multi-Modal and Multi-Model API for Real-Time Sign Language Translation*. <https://doi.org/10.1109/icccnt61001.2024.10724362>

- Kabir, M. H., Miah, A. S. M., Hadiuzzaman, M., & Shin, J. (2025). Combining state-of-the-art pre-trained deep learning models: A novel approach for Bangla Sign Language recognition using Max Voting Ensemble. *Systems and Soft Computing*, 7, 200230. <https://doi.org/10.1016/j.sasc.2025.200230>
- Myagila, K., Nyambo, D. G., & Dida, M. A. (2025). Two stream GRU model with ELU activation function for sign language recognition. *Intelligent Systems with Applications*, 26, 200513. <https://doi.org/10.1016/j.iswa.2025.200513>
- Guan, M., Wang, Y., Ma, G., Liu, J., & Sun, M. (2025). MSKA: Multi-stream keypoint attention network for sign language recognition and translation. *Pattern Recognition*, 165, 111602. <https://doi.org/10.1016/j.patcog.2025.111602>
- Wang, Z., Zhao, T., Ma, J., Chen, H., Liu, K., Shao, H., Wang, Q., & Ren, J. (2022). Hear Sign Language: A Real-Time End-to-End Sign Language Recognition System. *IEEE Transactions on Mobile Computing*, 21(7), 2398–2410. <https://doi.org/10.1109/TMC.2020.3038303>
- Sau, D., Dhol, S., K., M., & Jayavel, K. (2022). A Review on Real-Time Sign Language Recognition. *2022 International Conference on Computer Communication and Informatics (ICCCI)*, 1–5. <https://doi.org/10.1109/ICCCI54379.2022.9740868>
- Raval, J. J., & Gajjar, R. (2021). Real-time Sign Language Recognition using Computer Vision. *2021 3rd International Conference on Signal Processing and Communication (ICPSC)*, 542–546. <https://doi.org/10.1109/ICSPC51351.2021.9451709>
- Lee, C. K. M., Ng, K. K. H., Chen, C.-H., Lau, H. C. W., Chung, S. Y., & Tsoi, T. (2021). American sign language recognition and training method with recurrent neural network. *Expert Systems with Applications*, 167, 114403. <https://doi.org/10.1016/j.eswa.2020.114403>

Muthu Mariappan, H., & Gomathi, V. (2019). Real-Time Recognition of Indian Sign Language. *2019 International Conference on Computational Intelligence in Data Science (ICCIDS)*, 1–6. <https://doi.org/10.1109/ICCIDS.2019.8862125>

Taskiran, M., Killioglu, M., & Kahraman, N. (2018). A Real-Time System for Recognition of American Sign Language by using Deep Learning. *2018 41st International Conference on Telecommunications and Signal Processing (TSP)*, 1–5. <https://doi.org/10.1109/TSP.2018.8441304>

Likhar, P., Bhagat, N. K., & G N, R. (2020). Deep Learning Methods for Indian Sign Language Recognition. *2020 IEEE 10th International Conference on Consumer Electronics (ICCE-Berlin)*, 1–6. <https://doi.org/10.1109/ICCE-Berlin50680.2020.9352194>

Gupta, R., Gupta, K., Pandit, C., & Singh, P. (2024). Sign Language Recognition using VGG16 and ResNet50. *2024 11th International Conference on Computing for Sustainable Global Development (INDIACom)*, 996–1001. <https://doi.org/10.23919/INDIACom61295.2024.10498253>

Koonce, B. (2021). ResNet 50. In *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization* (pp. 63–72). Apress. https://doi.org/10.1007/978-1-4842-6168-2_6

Dong, K., Zhou, C., Ruan, Y., & Li, Y. (2020). MobileNetV2 Model for Image Classification. *2020 2nd International Conference on Information Technology and Compute Application (ITCA)*, 476–480. <https://doi.org/10.1109/ITCA52113.2020.00106>